MySQL Connector Python

5.1 Conectando-se ao MySQL usando Connector/Python
Referência:
https://dev.mysql.com/doc/connector-python/en/connector-python-example-connecting.html

O connect() construtor cria uma conexão com o servidor MySQL e retorna um MySQLConnection objeto.
O exemplo a seguir mostra como conectar-se ao servidor MySQL:

```
import mysql.connector

cnx = mysql.connector.connect(user='scott', password='password',
                              host='127.0.0.1',
                              database='employees')
cnx.close()
```

A Seção 7.1, “Argumentos de Conexão do Conector/Python”, descreve os argumentos de conexão permitidos.
Também é possível criar objetos de conexão usando a classe connection.MySQLConnection() :

```
from mysql.connector import (connection)

cnx = connection.MySQLConnection(user='scott', password='password',
                                 host='127.0.0.1',
                                 database='employees')
cnx.close()
```

Ambas as formas (usando o connect() construtor ou a classe diretamente) são válidas e funcionalmente equivalentes, mas o uso do construtor connect()é preferível e utilizado na maioria dos exemplos deste manual.
Para lidar com erros de conexão, use a try instrução e capture todos os erros usando a exceção `errors.Error` .

```
import mysql.connector
from mysql.connector import errorcode

try:
  cnx = mysql.connector.connect(user='scott',
                                database='employ')
except mysql.connector.Error as err:
  if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
    print("Something is wrong with your user name or password")
  elif err.errno == errorcode.ER_BAD_DB_ERROR:
    print("Database does not exist")
  else:
    print(err)
else:
  cnx.close()
```

Definir os argumentos de conexão em um dicionário e usar o **operador é outra opção:

```
import mysql.connector

config = {
  'user': 'scott',
  'password': 'password',
  'host': '127.0.0.1',
  'database': 'employees',
  'raise_on_warnings': True
}

cnx = mysql.connector.connect(**config)

cnx.close()
```

5.2 Criação de tabelas usando Connector/Python
Referência:
https://dev.mysql.com/doc/connector-python/en/connector-python-example-ddl.html

Todas as instruções DDL (Linguagem de Definição de Dados) são executadas usando uma estrutura de referência conhecida como cursor. Os exemplos a seguir mostram como criar as tabelas do Banco de Dados de Exemplo de Funcionários . Você precisará delas para os outros exemplos.
Em um servidor MySQL, as tabelas são objetos de longa duração e frequentemente acessadas por várias aplicações escritas em diferentes linguagens. Normalmente, você trabalha com tabelas já configuradas, em vez de criá-las dentro da sua própria aplicação. Evite criar e excluir tabelas repetidamente, pois essa é uma operação custosa. A exceção são as tabelas temporárias , que podem ser criadas e excluídas rapidamente dentro de uma aplicação.

```
from __future__ import print_function

import mysql.connector
from mysql.connector import errorcode

DB_NAME = 'employees'

TABLES = {}
TABLES['employees'] = (
    "CREATE TABLE `employees` ("
    "  `emp_no` int(11) NOT NULL AUTO_INCREMENT,"
    "  `birth_date` date NOT NULL,"
    "  `first_name` varchar(14) NOT NULL,"
    "  `last_name` varchar(16) NOT NULL,"
    "  `gender` enum('M','F') NOT NULL,"
    "  `hire_date` date NOT NULL,"
    "  PRIMARY KEY (`emp_no`)"
    ") ENGINE=InnoDB")
```

```
TABLES['departments'] = (
    "CREATE TABLE `departments` ("
    "  `dept_no` char(4) NOT NULL,"
    "  `dept_name` varchar(40) NOT NULL,"
    "  PRIMARY KEY (`dept_no`), UNIQUE KEY `dept_name` (`dept_name`)"
    ") ENGINE=InnoDB")

TABLES['salaries'] = (
    "CREATE TABLE `salaries` ("
    "  `emp_no` int(11) NOT NULL,"
    "  `salary` int(11) NOT NULL,"
    "  `from_date` date NOT NULL,"
    "  `to_date` date NOT NULL,"
    "  PRIMARY KEY (`emp_no`,`from_date`), KEY `emp_no` (`emp_no`),"
    "  CONSTRAINT `salaries_ibfk_1` FOREIGN KEY (`emp_no`) "
    "     REFERENCES `employees` (`emp_no`) ON DELETE CASCADE"
    ") ENGINE=InnoDB")

TABLES['dept_emp'] = (
    "CREATE TABLE `dept_emp` ("
    "  `emp_no` int(11) NOT NULL,"
    "  `dept_no` char(4) NOT NULL,"
    "  `from_date` date NOT NULL,"
    "  `to_date` date NOT NULL,"
    "  PRIMARY KEY (`emp_no`,`dept_no`), KEY `emp_no` (`emp_no`),"
    "  KEY `dept_no` (`dept_no`),"
    "  CONSTRAINT `dept_emp_ibfk_1` FOREIGN KEY (`emp_no`) "
    "     REFERENCES `employees` (`emp_no`) ON DELETE CASCADE,"
    "  CONSTRAINT `dept_emp_ibfk_2` FOREIGN KEY (`dept_no`) "
    "     REFERENCES `departments` (`dept_no`) ON DELETE CASCADE"
    ") ENGINE=InnoDB")

TABLES['dept_manager'] = (
    "  CREATE TABLE `dept_manager` ("
    "  `emp_no` int(11) NOT NULL,"
    "  `dept_no` char(4) NOT NULL,"
    "  `from_date` date NOT NULL,"
    "  `to_date` date NOT NULL,"
    "  PRIMARY KEY (`emp_no`,`dept_no`),"
    "  KEY `emp_no` (`emp_no`),"
    "  KEY `dept_no` (`dept_no`),"
    "  CONSTRAINT `dept_manager_ibfk_1` FOREIGN KEY (`emp_no`) "
    "     REFERENCES `employees` (`emp_no`) ON DELETE CASCADE,"
    "  CONSTRAINT `dept_manager_ibfk_2` FOREIGN KEY (`dept_no`) "
    "     REFERENCES `departments` (`dept_no`) ON DELETE CASCADE"
    ") ENGINE=InnoDB")
```

```
TABLES['titles'] = (
    "CREATE TABLE `titles` ("
    "  `emp_no` int(11) NOT NULL,"
    "  `title` varchar(50) NOT NULL,"
    "  `from_date` date NOT NULL,"
    "  `to_date` date DEFAULT NULL,"
    "  PRIMARY KEY (`emp_no`,`title`,`from_date`), KEY `emp_no` (`emp_no`),"
    "  CONSTRAINT `titles_ibfk_1` FOREIGN KEY (`emp_no`)"
    "     REFERENCES `employees` (`emp_no`) ON DELETE CASCADE"
    ") ENGINE=InnoDB")
```

O código anterior mostra como armazenamos as CREATE declarações em um dicionário Python chamado `declarations` TABLES. Também definimos o banco de dados em uma variável global chamada `database` DB_NAME, o que permite usar facilmente um esquema diferente.

```
cnx = mysql.connector.connect(user='scott')
cursor = cnx.cursor()
```

Um único servidor MySQL pode gerenciar vários bancos de dados . Normalmente, você especifica o banco de dados para o qual deseja alternar ao se conectar ao servidor MySQL. Este exemplo não se conecta ao banco de dados ao estabelecer a conexão, para que possa verificar se o banco de dados existe e criá-lo caso contrário.

```
def create_database(cursor):
    try:
        cursor.execute(
            "CREATE DATABASE {} DEFAULT CHARACTER SET 'utf8'".format(DB_NAME))
    except mysql.connector.Error as err:
        print("Failed creating database: {}".format(err))
        exit(1)

try:
    cursor.execute("USE {}".format(DB_NAME))
except mysql.connector.Error as err:
    print("Database {} does not exists.".format(DB_NAME))
    if err.errno == errorcode.ER_BAD_DB_ERROR:
        create_database(cursor)
        print("Database {} created successfully.".format(DB_NAME))
        cnx.database = DB_NAME
    else:
        print(err)
        exit(1)
```

Primeiro, tentamos mudar para um banco de dados específico usando a database propriedade do objeto de conexão cnx. Se ocorrer um erro, examinamos o número do erro para verificar se o banco de dados não existe. Caso exista, chamamos a create_database função para criá-lo.
Em caso de qualquer outro erro, o aplicativo é encerrado e exibe a mensagem de erro.

Após criarmos ou alterarmos com sucesso o banco de dados de destino, criamos as tabelas iterando sobre os itens do TABLES dicionário:

```
for table_name in TABLES:
    table_description = TABLES[table_name]
    try:
        print("Creating table {}: ".format(table_name), end='')
        cursor.execute(table_description)
    except mysql.connector.Error as err:
        if err.errno == errorcode.ER_TABLE_EXISTS_ERROR:
            print("already exists.")
        else:
            print(err.msg)
    else:
        print("OK")

cursor.close()
cnx.close()
```

Para lidar com o erro quando a tabela já existe, notificamos o usuário de que ela já estava lá. Outros erros são exibidos, mas continuamos criando tabelas. (O exemplo mostra como lidar com a condição " tabela já existe " para fins ilustrativos. Em uma aplicação real, normalmente evitaríamos completamente a condição de erro usando a IF NOT EXISTS cláusula `WHERE` da CREATE TABLE instrução.)

O resultado seria algo como isto:

```
Database employees does not exists.
Database employees created successfully.
Creating table employees: OK
Creating table departments: already exists.
Creating table salaries: already exists.
Creating table dept_emp: OK
Creating table dept_manager: OK
Creating table titles: OK
```

5.3 Inserindo dados usando Connector/Python

Referência:
https://dev.mysql.com/doc/connector-python/en/connector-python-example-cursor-transaction.html

A inserção ou atualização de dados também é feita usando a estrutura de manipulador conhecida como cursor. Quando você usa um mecanismo de armazenamento transacional como InnoDB o MySQL (padrão no MySQL 5.5 e versões superiores), você deve confirmar os dados após uma sequência de instruções INSERT INSERT, INSERT DELETE e UPDATE POST.
Este exemplo mostra como inserir novos dados. O segundo INSERT depende do valor da chave primária recém-criada do primeiro. O exemplo também demonstra como usar formatos estendidos. A tarefa é adicionar um novo funcionário que começará a trabalhar amanhã com um salário de 50.000.

Observação

O exemplo a seguir utiliza tabelas criadas na Seção 5.2, “Criando Tabelas Usando Connector/Python” . A AUTO_INCREMENTopção de coluna para a chave primária da employeestabela é importante para garantir dados confiáveis e facilmente pesquisáveis.

```
from __future__ import print_function
from datetime import date, datetime, timedelta
import mysql.connector

cnx = mysql.connector.connect(user='scott', database='employees')
cursor = cnx.cursor()

tomorrow = datetime.now().date() + timedelta(days=1)

add_employee = ("INSERT INTO employees "
               "(first_name, last_name, hire_date, gender, birth_date) "
               "VALUES (%s, %s, %s, %s, %s)")
add_salary = ("INSERT INTO salaries "
              "(emp_no, salary, from_date, to_date) "
              "VALUES (%(emp_no)s, %(salary)s, %(from_date)s, %(to_date)s)")

data_employee = ('Geert', 'Vanderkelen', tomorrow, 'M', date(1977, 6, 14))

# Insert new employee
cursor.execute(add_employee, data_employee)
emp_no = cursor.lastrowid

# Insert salary information
data_salary = {
  'emp_no': emp_no,
  'salary': 50000,
  'from_date': tomorrow,
  'to_date': date(9999, 1, 1),
}
cursor.execute(add_salary, data_salary)

# Make sure data is committed to the database
cnx.commit()

cursor.close()
cnx.close()
```

Primeiro, abrimos uma conexão com o servidor MySQL e armazenamos o objeto de conexão na variável cnx. Em seguida, criamos um novo cursor, por padrão um objeto MySQLCursor cursor() , usando o método da conexão.
Poderíamos calcular o valor de amanhã chamando uma função do banco de dados, mas, para maior clareza, faremos isso em Python usando o datetime módulo.

Ambas INSERTas declarações são armazenadas nas variáveis chamadas `x` add_employeee `y` add_salary. Observe que a segunda INSERT declaração usa códigos de formato Python estendidos.
As informações do novo funcionário são armazenadas na tupla data_employee. A consulta para inserir o novo funcionário é executada e recuperamos o valor recém-inserido para a emp_nocoluna (uma AUTO_INCREMENT coluna) usando a lastrowid propriedade do objeto cursor.
Em seguida, inserimos o novo salário do novo funcionário, usando a emp_no variável no dicionário que contém os dados. Esse dicionário é passado para o execute()método do objeto cursor caso ocorra um erro.
Como o Connector/Python desativa o autocommit por padrão , e o MySQL 5.5 e versões superiores usam InnoDB tabelas transacionais por padrão, é necessário confirmar suas alterações usando o commit()método da conexão. Você também pode reverter as alterações usando o rollback()método.

5.4 Consultando dados usando Connector/Python
Referência:
https://dev.mysql.com/doc/connector-python/en/connector-python-example-cursor-select.html

O exemplo a seguir mostra como consultar dados usando um cursor criado com o cursor() método da conexão. Os dados retornados são formatados e exibidos no console.
A tarefa consiste em selecionar todos os funcionários contratados no ano de 1999 e imprimir seus nomes e datas de contratação no console.

```
import datetime
import mysql.connector

cnx = mysql.connector.connect(user='scott', database='employees')
cursor = cnx.cursor()

query = ("SELECT first_name, last_name, hire_date FROM employees "
         "WHERE hire_date BETWEEN %s AND %s")

hire_start = datetime.date(1999, 1, 1)
hire_end = datetime.date(1999, 12, 31)

cursor.execute(query, (hire_start, hire_end))

for (first_name, last_name, hire_date) in cursor:
  print("{}, {} was hired on {:%d %b %Y}".format(
    last_name, first_name, hire_date))

cursor.close()
cnx.close()
```

Primeiro, abrimos uma conexão com o servidor MySQL e armazenamos o objeto de conexão na variável cnx. Em seguida, criamos um novo cursor, por padrão um objeto MySQLCursor cursor() , usando o método da conexão.

No exemplo anterior, armazenamos a SELECT declaração na variável query. Observe que estamos usando %s marcadores sem aspas onde deveriam estar as datas. O Connector/Python converte hire_starte hire_endde

tipos Python para um tipo de dados que o MySQL entende e adiciona as aspas necessárias. Neste caso, ele substitui o primeiro %s por '1999-01-01'e o segundo por '1999-12-31'.

Em seguida, executamos a operação armazenada na queryvariável usando o execute() método. Os dados usados para substituir os %smarcadores na consulta são passados como uma tupla: (hire_start, hire_end).

Após a execução da consulta, o servidor MySQL está pronto para enviar os dados. O conjunto de resultados pode conter zero linhas, uma linha ou 100 milhões de linhas. Dependendo do volume esperado, você pode usar diferentes técnicas para processar esse conjunto de resultados. Neste exemplo, usamos o cursorobjeto como um iterador. A primeira coluna da linha é armazenada na variável `first` first_name, a segunda em `second` last_namee a terceira em `third` hire_date.

Imprimimos o resultado, formatando a saída usando a função integrada do Python format(). Observe que o hire_datevalor foi convertido automaticamente pelo Connector/Python em um datetime.dateobjeto Python. Isso significa que podemos formatar facilmente a data em um formato mais legível para humanos.

O resultado deverá ser algo como isto:

```
..
Wilharm, LiMin was hired on 16 Dec 1999
Wielonsky, Lalit was hired on 16 Dec 1999
Kamble, Dannz was hired on 18 Dec 1999
DuBourdieux, Zhongwei was hired on 19 Dec 1999
Fujisawa, Rosita was hired on 20 Dec 1999
..
```